SESSION 04 / PRINTABLE GUIDE

Main lab: Explore, clean, and publish realistic notebook datasets

Session 04 | Primary participant workflow
Output: Full-batch quality evidence, useful notebook charts, explicit Silver contracts, and verified transfers for Warehouse Gold.

Mandatory prerequisite: The customer and facilitator must complete pre-delivery setup before the delivery day. The presenter must verify completion with the customer 3-5 business days beforehand and reconfirm on the preceding business day. This lab starts with prepared data, permissions, connections, and Warehouse objects.

For first-time users: Start with Notebook 101 and its repair exercise, then import the main-lab starter. It includes the complete pipeline plus three short practice checkpoints, not a blank notebook to assemble. The solution fills those practice cells. See import/run instructions.

Choose the right tier

Use this lab for the main end-to-end session. The small CSV and JSON fixtures remain useful for stepping through lessons 02-07 one row at a time; they are not the main analytical workload. Do not run their four-/nine-row assertions or fixture-specific Warehouse scripts against these larger datasets.

The main sources contain 50 devices over fourteen days and 5,000 nested orders. They demonstrate multi-file ingestion, sampled versus whole-batch exploration, repeatable transformations, controlled quarantine, and useful serving outputs. Their size alone does not require Spark; equivalent SQL remains a valid choice when the surrounding requirements favor it.

1. Prepare and load the batch contract

Use an approved, isolated default Lakehouse and Warehouse per participant/pair. Confirm the mandatory setup is signed off: the ADLS source files have already been copied to `Files/session_04/scenario/devices` and `Files/session_04/scenario/orders`, with `manifest.json` and `expected/` staged separately under `Files/session_04/scenario`. Preserve that structure; do not combine it with the micro-fixture inputs. Do not start first-time provisioning or uploads during delivery. An approved manual-upload fallback must have been agreed and labeled before class; it does not prove ADLS ingestion.

Use the imported Fabric PySpark starter, or run the Python blocks below as separate cells in order. The handout shows the worked practice answers. Spark performs the source transformations; only the small answer-key manifest and bounded quality summaries are collected to the driver.

Review the write-approval gate

The notebook replaces known `s04_lab_` Lakehouse training snapshots later in the flow. Confirm the facilitator-approved default Lakehouse and target scope before enabling the first cell. Both starter and solution **stop here by default**. This explicit safety stop is not one of the troubleshooting bugs. Do not enable it against a shared or production target.

PYTHON

```python
RUN_TRAINING_LAB = False
if not RUN_TRAINING_LAB:
    raise RuntimeError(
        "Safety stop: review the s04_lab_ writes and confirm the assigned "
        "default Lakehouse with the facilitator, then set RUN_TRAINING_LAB=True."
    )
```

Changing the flag is approval to run the known training snapshot workflow, not proof that the current Lakehouse is correct. The user must verify that context. After approval, continue with the setup cell:

PYTHON

```python
import json
from decimal import Decimal
from pyspark.sql import functions as F, types as T, Window

BASE = "Files/session_04/scenario"
spark.conf.set("spark.sql.session.timeZone", "UTC")
manifest = json.loads(
    spark.read.option("wholetext", True).text(BASE + "/manifest.json").first()[0]
)
if manifest["dataset_version"] != "s04-scenario-v1":
    raise ValueError("Use the matching versioned lab inputs and answer key.")
D = manifest["devices"]
O = manifest["orders"]


def normalized(name):
    value = F.upper(F.trim(F.col(name)))
    return F.when(F.length(value) > 0, value)


def check_reasons(rejected, expected):
    actual = {
        row["quality_issue"]: row["count"]
        for row in rejected.groupBy("quality_issue").count().collect()
    }
    if actual != expected:
        raise ValueError(f"Unexpected primary rejection counts: {actual}")
```

```
def check_same(actual, expected, columns):
    left, right = actual.select(*columns), expected.select(*columns)
    if (
        left.exceptAll(right).limit(1).count()
        or right.exceptAll(left).limit(1).count()
    ):
        raise ValueError("Actual and expected business rows differ.")
```

The expected files are a synthetic teaching oracle, not inferred production thresholds. Before upload, the maintainer's offline validator checks their hashes, independently recomputes them from the sources, and verifies deterministic regeneration. An altered source requires a reviewed new dataset version and matching expectations, not simply weaker assertions.

2. Land device Bronze without guessing types

PYTHON

```
device_fields = [
    "device_id", "event_id", "event_time_utc", "temperature_c",
    "firmware_version", "signal_strength_dbm", "ui_theme", "debug_message",
    "export_row_number", "site_id", "device_type",
]
device_schema = T.StructType([
    T.StructField(name, T.StringType()) for name in device_fields
])
devices = (
    spark.read.schema(device_schema).option("header", True)
    .option("mode", "FAILFAST").csv(BASE + "/devices/*.csv")
    .withColumn("source_file", F.input_file_name())
    .withColumn("batch_id", F.lit(D["batch_id"]))
    .withColumn("ingested_at_utc", F.current_timestamp())
)
devices.write.format("delta").mode("overwrite").saveAsTable("s04_lab_device_bronze")
devices = spark.table("s04_lab_device_bronze")
if devices.count() != D["raw_rows"]:
    raise ValueError("The complete device source scope was not landed.")
devices.printSchema()
devices.orderBy("event_time_utc", "device_id").limit(20).show(truncate=False)
```

The CSV header/order must match the eleven-field contract in every source file. CSV quoting preserves the comma in some debug messages. `FAILFAST` detects parser problems, not business-invalid values such as `sensor error`.

All `s04_lab_` Lakehouse tables here are overwritten current-batch snapshots. Use a fresh isolated target; incompatible prior schemas require a reviewed migration or a new training target, not blanket schema overwrite.

3. Explore first; use Data Wrangler on a labeled sample

PYTHON

```
missing = devices.agg(*[
    F.sum(F.when(
        F.col(name).isNull() | (F.length(F.trim(F.col(name))) == 0), 1
    ).otherwise(0)).alias(name)
    for name in device_fields
])
display(missing)
display(devices.groupBy("site_id", "device_type", "firmware_version").count())
devices.agg(
    F.countDistinct("device_id").alias("raw_spellings"),
    F.countDistinct(F.upper(F.trim("device_id"))).alias("normalized_devices"),
).show()
display(
    devices.withColumn("temperature_value", F.expr(
        "try_cast(temperature_c as decimal(8,2))"
    )).select("temperature_value").summary(
        "count", "min", "25%", "50%", "75%", "max"
    )
)

wrangler_sample = devices.sample(fraction=0.03, seed=42).limit(5000)
display(wrangler_sample)
```

Whole-batch expectations include 100 raw spellings for 50 devices, 1,005 missing event IDs, 1,005 blank temperatures, and 1,002 missing signal values. Invalid timestamps and `sensor error` are **present**, not missing. The 200-degree values are parseable but outside the approved range. Samples may miss these conditions.

When the kernel is idle, open `wrangler_sample` in Data Wrangler using the review/export workflow. Record the actual sample size and method; a seeded fraction plus cap is not guaranteed representative. Preview trimming/uppercasing identifiers, a safe signal-strength conversion, and removal of UI/debug/export-position fields from a **candidate**, not Bronze.

Export and inspect the actual generated PySpark. Apply the reviewed function to `devices`, not just the sample, and confirm 201,579 candidate rows remain. Missing optional signal must stay null, not zero; version labels must stay text. Do not fill temperatures with averages or drop duplicates. Then resume the authoritative rules below from unchanged Bronze; the candidate is an experiment, not an alternate acceptance implementation. If Wrangler is unavailable, label that UI step blocked rather than claiming the PySpark code proves UI support.

Practice checkpoint 1: Investigate firmware at one site

Filter the complete `devices` DataFrame to S03, group by firmware version, and count source rows. Label these as raw rows, not accepted measurements. This is a Spark transformation, not a Python loop over collected records.

<!-- practice: main-firmware -->

PYTHON

```python
firmware_profile = (
    devices.filter(F.col("site_id") == "S03")
    .groupBy("firmware_version").count().orderBy("firmware_version")
)
display(firmware_profile)
```

4. Normalize, classify, and persist device Silver

PYTHON

```python
device_keys = Window.partitionBy("batch_id", "device_id", "event_id")
prepared = (
    devices.withColumn("device_id", normalized("device_id"))
    .withColumn("event_id", normalized("event_id"))
    .withColumn("event_time_text", F.trim("event_time_utc"))
    .withColumn("temperature_text", F.trim("temperature_c"))
    .withColumn("event_time", F.expr("try_cast(event_time_text as timestamp)"))
    .withColumn("temperature", F.expr(
        "try_cast(temperature_text as decimal(8,2))"
    ))
)
evaluated_devices = (
    prepared.withColumn("event_key_count", F.count("*").over(device_keys))
    .withColumn("quality_issue",
        F.when(F.col("device_id").isNull() | F.col("event_id").isNull(),
               "MISSING_EVENT_KEY")
        .when(F.col("event_key_count") > 1, "DUPLICATE_EVENT")
        .when(
            F.col("event_time").isNull()
            | ~F.col("event_time_text").rlike(
                r"^[0-9]{4}-[0-9]{2}-[0-9]{2}T[0-9]{2}:[0-9]{2}:[0-9]{2}Z$"
            ), "INVALID_TIMESTAMP")
        .when(
            F.col("temperature").isNull()
            | ~F.col("temperature_text").rlike(r"^-?[0-9]{1,3}(\.[0-9]{1,2})?$"),
            "INVALID_TEMPERATURE")
        .when(~F.col("temperature").between(-40, 85), "OUT_OF_RANGE")
        .when(
            F.col("site_id").isNull() | ~F.col("site_id").rlike(r"^S[0-9]{2}$")
            | F.col("device_type").isNull()
            | ~F.col("device_type").isin("AMBIENT", "COLD_STORE", "PUMP"),
            "MISSING_DEVICE_CONTEXT")
    )
)
accepted_devices = evaluated_devices.filter(F.col("quality_issue").isNull())
```

```
rejected_devices = evaluated_devices.filter(F.col("quality_issue").isNotNull())
if (accepted_devices.count(), rejected_devices.count()) != (
    D["accepted_rows"], D["rejected_rows"]
):
    raise ValueError("Unexpected device acceptance counts.")
check_reasons(rejected_devices, D["rejection_counts"])

device_silver = (
    accepted_devices.select(
        "batch_id", "device_id", "event_id",
        F.col("event_time").alias("event_time_utc"),
        F.col("temperature").alias("temperature_c"),
        "site_id", "device_type", "firmware_version",
        F.expr("try_cast(signal_strength_dbm as int)").alias("signal_strength_dbm"),
        "source_file", "ingested_at_utc",
    ).withColumn("quality_rule_version", F.lit("s04-lab-devices-v1"))
)
device_silver.write.format("delta").mode("overwrite").saveAsTable("s04_lab_device_silver")
rejected_devices.write.format("delta").mode("overwrite").saveAsTable("s04_lab_device_rejected")
```

Expected: **194,545 accepted events; 7,034 rejected rows**. Primary rejections: 2,010 duplicate occurrences, 2,010 invalid temperatures, 1,004 invalid timestamps, 1,005 missing keys, and 1,005 out-of-range readings. Both occurrences of each repeated event key are rejected. Missing-key precedence prevents blank IDs being misreported as ordinary duplicate events.

Silver's twelve columns keep event facts, site/type, diagnostic context, and lineage. Raw text, debug/UI fields, export position, and quality helpers remain in Bronze/quarantine or working state. Optional signal values may be null by contract; that does not make them good signal.

5. Prepare and verify daily Gold rows

PYTHON

```
device_silver = spark.table("s04_lab_device_silver")
daily = (
    device_silver.groupBy(
        "batch_id", F.to_date("event_time_utc").alias("reading_date"),
        "device_id", "site_id", "device_type",
    ).agg(
        F.count("*").alias("reading_count"),
        F.min("temperature_c").alias("min_temperature_c"),
        F.max("temperature_c").alias("max_temperature_c"),
        F.avg("temperature_c").cast("decimal(8,2)").alias("avg_temperature_c"),
    )
)
daily_schema = (
    T.StructType().add("batch_id", "string").add("reading_date", "date")
```

```
    .add("device_id", "string").add("site_id", "string").add("device_type", "string")
    .add("reading_count", "long").add("min_temperature_c", "decimal(8,2)")
    .add("max_temperature_c", "decimal(8,2)").add("avg_temperature_c", "decimal(8,2)")
)
expected_daily = (
    spark.read.schema(daily_schema).option("header", True)
    .option("mode", "FAILFAST").csv(BASE + "/expected/device_daily.csv")
)
if daily.count() != D["daily_rows"]:
    raise ValueError("Expected 700 device/day rows.")
if daily.agg(F.sum("reading_count")).first()[0] != D["accepted_rows"]:
    raise ValueError("Daily output must account for every accepted event.")
check_same(daily, expected_daily, daily_schema.fieldNames())
daily.select(*daily_schema.fieldNames()).write.format("delta").mode(
    "overwrite"
).saveAsTable("s04_lab_device_daily_transfer")
```

This compares **all 700 business rows in both directions**, not a preview or just a global mean. Site and device type are stable for each device in this source contract; firmware can change within a day and is not forced into one arbitrary daily value. A type/site change requires a reviewed new grain or effective-dated mapping, not silent fragmentation of device/day rows.

6. Answer questions with notebook visualizations

These are full-scope Spark aggregates suitable for the Fabric `display` chart controls. Do not collect 200,000 source rows to pandas for plotting.

PYTHON

```
display(rejected_devices.groupBy("quality_issue").count().orderBy("quality_issue"))

hourly = (
    device_silver.filter(F.col("device_id").isin("D021", "D012"))
    .groupBy("device_id", F.date_trunc("hour", "event_time_utc").alias("hour_utc"))
    .agg(
        F.avg("temperature_c").alias("avg_temperature_c"),
        F.count("*").alias("accepted_readings"),
    ).orderBy("hour_utc", "device_id")
)
display(hourly)

coverage = (
    daily.withColumn("scheduled_readings", F.lit(288))
    .withColumn("accepted_slot_shortfall", 288 - F.col("reading_count"))
    .withColumn("accepted_percent", F.round(F.col("reading_count") / 288 * 100, 2))
)
display(coverage.orderBy("reading_date", "device_id"))
```

```
display(
    device_silver.groupBy("site_id", "device_type", "firmware_version")
    .agg(
        F.count("*").alias("accepted_readings"),
        F.avg("temperature_c").alias("avg_temperature_c"),
    ).orderBy("site_id", "device_type", "firmware_version")
)
```

Create a rejection bar chart, a two-series hourly line chart (X = `hour_utc`, Y = `avg_temperature_c`, series = `device_id`), and a coverage chart filtered to D017. Hourly output has at most 672 rows; coverage has 700. Chart controls must use the grouped metric, not count summary rows or mix multiple dates into unlabeled device bars.

Investigate the S03/firmware association around October 7 noon. D021 and D012 are both pump devices at different sites, but even this comparison does not control all confounders. The generator injects the association; it is not causal evidence. An hourly mean is based on accepted readings, not necessarily twelve complete observations.

There are **1,026 missing source slots**, **7,034 rejected source rows**, and **7,055 scheduled slots without an accepted reading**. These differ because quarantine includes both copies of duplicated events. The generated schedule is known, so 288 is an explicit device/day denominator here. Do not interpret the accepted shortfall as device downtime; it includes data-quality rejection.

Practice checkpoint 2: Inspect one device/day

Use the prepared `coverage` DataFrame to inspect D017 on October 7. Show accepted count and accepted-slot shortfall. Explain why the shortfall is not identical to source omissions, quarantine rows, or device downtime.

<!-- practice: main-coverage -->

PYTHON

```
device_day = coverage.filter(
    (F.col("device_id") == "D017")
    & (F.col("reading_date") == F.lit("2026-10-07").cast("date"))
).select("reading_date", "device_id", "reading_count", "accepted_slot_shortfall")
display(device_day)
```

7. Land and expand nested order documents

PYTHON

```
line_schema = (
    T.StructType().add("line_number", "string").add("sku", "string")
```

```
    .add("quantity", "string").add("unit_price", "string")
    .add("product", T.StructType().add("category", "string").add("description", "string"))
    .add("display_label", "string")
)
order_schema = (
    T.StructType().add("order_id", "string").add("order_date", "string")
    .add("order_status", "string")
    .add("customer", T.StructType().add("id", "string").add("name", "string").add("region", "strin
g"))
    .add("shipping", T.StructType()
         .add("address", T.StructType().add("city", "string").add("country", "string"))
         .add("service_level", "string"))
    .add("lines", T.ArrayType(line_schema))
    .add("ui_theme", "string").add("debug_message", "string")
)
orders = (
    spark.read.text(BASE + "/orders/*.jsonl")
    .select(F.col("value").alias("raw_json"), F.input_file_name().alias("source_file"))
    .withColumn("batch_id", F.lit(O["batch_id"]))
    .withColumn("ingested_at_utc", F.current_timestamp())
)
orders.write.format("delta").mode("overwrite").saveAsTable("s04_lab_order_bronze")
orders = spark.table("s04_lab_order_bronze")
if orders.count() != O["documents"]:
    raise ValueError("Expected 5,000 source order documents.")
parsed = orders.withColumn("payload", F.from_json("raw_json", order_schema))
display(parsed.select("payload").limit(10))

flat = (
    parsed.select(
        "batch_id", "source_file", "raw_json", "ingested_at_utc",
        normalized("payload.order_id").alias("order_id"),
        F.trim("payload.order_date").alias("order_date_text"),
        normalized("payload.order_status").alias("order_status"),
        normalized("payload.customer.id").alias("customer_id"),
        F.trim("payload.customer.name").alias("customer_name"),
        normalized("payload.customer.region").alias("customer_region"),
        F.col("payload.shipping.service_level").alias("shipping_service_level"),
        F.explode_outer("payload.lines").alias("item"),
    )
    .withColumn("order_date", F.expr("try_cast(order_date_text as date)"))
    .withColumn("line_number", F.expr("try_cast(item.line_number as int)"))
    .withColumn("sku", normalized("item.sku"))
    .withColumn("product_category", normalized("item.product.category"))
    .withColumn("quantity_text", F.trim("item.quantity"))
    .withColumn("price_text", F.trim("item.unit_price"))
    .withColumn("quantity", F.expr("try_cast(quantity_text as int)"))
    .withColumn("unit_price", F.expr("try_cast(price_text as decimal(18,2))"))
)
if flat.count() != O["expanded_rows"]:
```

```
    raise ValueError("Unexpected expansion scope.")
display(flat.groupBy("customer_region", "order_status").count())
```

Explore `size(payload.lines)`, optional shipping, and product descriptions before choosing columns. Predict why 5,000 documents become 15,054 evidence rows. `explode_outer` retains 50 empty-order markers; these are not real order lines. There are 1,000 documents without shipping, which is **allowed**, not a reason to reject their lines. Missing required customer/category/date is different.

8. Apply order rules and verify every accepted transfer row

PYTHON

```
order_keys = Window.partitionBy("batch_id", "order_id", "line_number")
evaluated_orders = (
    flat.withColumn("line_key_count", F.count("*").over(order_keys))
    .withColumn("quality_issue",
        F.when(F.col("order_id").isNull(), "MISSING_ORDER_ID")
        .when(F.col("line_number").isNull() | (F.col("line_number") <= 0), "NO_VALID_LINE")
        .when(F.col("line_key_count") > 1, "DUPLICATE_LINE")
        .when(
            F.col("customer_id").isNull() | F.col("customer_name").isNull()
            | (F.length("customer_name") == 0) | F.col("customer_region").isNull(),
            "MISSING_CUSTOMER")
        .when(
            F.col("order_date").isNull()
            | ~F.col("order_date_text").rlike(r"^[0-9]{4}-[0-9]{2}-[0-9]{2}$"),
            "INVALID_ORDER_DATE")
        .when(
            F.col("order_status").isNull()
            | ~F.col("order_status").isin("NEW", "PROCESSING", "SHIPPED", "CANCELLED"),
            "INVALID_ORDER_STATUS")
        .when(F.col("sku").isNull(), "MISSING_SKU")
        .when(F.col("product_category").isNull(), "MISSING_PRODUCT_CATEGORY")
        .when(
            F.col("quantity").isNull()
            | ~F.col("quantity_text").rlike(r"^[1-9][0-9]{0,5}$"), "INVALID_QUANTITY")
        .when(
            F.col("unit_price").isNull()
            | ~F.col("price_text").rlike(r"^[0-9]{1,8}(\.[0-9]{1,2})?$"), "INVALID_PRICE")
    )
    .withColumn("line_amount", (F.col("quantity") * F.col("unit_price")).cast("decimal(18,2)"))
)
accepted_orders = evaluated_orders.filter(F.col("quality_issue").isNull())
rejected_orders = evaluated_orders.filter(F.col("quality_issue").isNotNull())
if (accepted_orders.count(), rejected_orders.count()) != (
    O["accepted_rows"], O["rejected_rows"]
):
    raise ValueError("Unexpected order acceptance counts.")
```

```
check_reasons(rejected_orders, O["rejection_counts"])
if accepted_orders.agg(F.sum("line_amount")).first()[0] != Decimal(O["accepted_line_amount"]):
    raise ValueError("Unexpected accepted-line amount.")

order_fields = [
    "batch_id", "order_id", "line_number", "order_date", "order_status",
    "customer_id", "customer_name", "customer_region", "sku", "product_category",
    "quantity", "unit_price", "line_amount",
]
order_silver = (
    accepted_orders.select(
        *order_fields, "shipping_service_level", "source_file", "ingested_at_utc"
    ).withColumn("quality_rule_version", F.lit("s04-lab-orders-v1"))
)
order_silver.write.format("delta").mode("overwrite").saveAsTable("s04_lab_order_silver")
rejected_orders.write.format("delta").mode("overwrite").saveAsTable("s04_lab_order_rejected")

order_transfer = spark.table("s04_lab_order_silver").select(*order_fields)
order_types = {
    "line_number": "int", "order_date": "date", "quantity": "int",
    "unit_price": "decimal(18,2)", "line_amount": "decimal(18,2)",
}
expected_order_schema = T.StructType()
for name in order_fields:
    expected_order_schema.add(name, order_types.get(name, "string"))
expected_orders = (
    spark.read.schema(expected_order_schema).option("header", True)
    .option("mode", "FAILFAST").csv(BASE + "/expected/order_lines.csv")
)
check_same(order_transfer, expected_orders, order_fields)
order_transfer.write.format("delta").mode("overwrite").saveAsTable("s04_lab_order_line_transfer")
```

Expected: **14,391 accepted lines, 663 rejected evidence rows, 5,643,227.95 in accepted-line amount**. The manifest lists all eight rejection counts; compare them, not just the total. Full transfer comparison checks keys, dimensions, dates, quantities, prices, and amounts for every accepted line.

Silver has seventeen columns: thirteen serving fields, optional shipping service, and three lineage fields. Product description, shipping address, UI/debug fields, raw payloads, and helpers stay out of this serving contract. This is a scoped analytical decision, not a statement that addresses/descriptions never matter.

The steward explicitly approves publishing accepted lines while retaining rejected evidence. A complete-order ledger would need a different policy: rejecting one line could block the entire order or batch. Cancelled orders are retained with their status; do not call their line amounts recognized revenue or silently exclude them from validation totals.

Practice checkpoint 3: Summarize accepted lines by status

Read only the prepared `order_transfer` DataFrame. Produce accepted-line count and amount by status, including cancelled orders. Explain why neither metric is automatically recognized revenue.

<!-- practice: main-status -->

PYTHON

```python
status_summary = order_transfer.groupBy("order_status").agg(
    F.count("*").alias("accepted_lines"),
    F.sum("line_amount").alias("accepted_line_amount"),
).orderBy("order_status")
display(status_summary)
```

9. Copy, validate, and publish Warehouse Gold

Use the transfer controls and tenant checks in lesson 04, but use **only the scenario-specific SQL below** for this tier.

1. Confirm the facilitator already ran scenario Warehouse setup during pre-delivery preparation in the isolated **Warehouse**, not the Lakehouse SQL analytics endpoint. Do not rerun its CREATE statements over existing targets; missing or incompatible objects are a readiness blocker.
2. After the whole notebook succeeds, review the batch, all rejection counts, accepted-line policy, and complete expected-row comparisons.
3. Use the two prepared copy activities and verify the mappings below. The appropriate batch-scoped staging cleanup must already be configured as each activity's reviewed pre-copy script.
4. Require successful copy and matching staging scope. Run order publication and device publication separately.

Source Lakehouse table	Warehouse staging	Explicit mapping	Expected rows
`s04_lab_order_line_transfer`	`stg.s04_lab_order_line`	All thirteen fields in `order_fields`, by name	14,391
`s04_lab_device_daily_transfer`	`stg.s04_lab_device_daily`	All nine fields in `daily_schema`, by name	700, representing 194,545 accepted events

SQL

```sql
-- Orders activity only:
DELETE FROM stg.s04_lab_order_line WHERE batch_id = 's04-lab-orders-001';

-- Devices activity only:
DELETE FROM stg.s04_lab_device_daily WHERE batch_id = 's04-lab-devices-001';
```

Never clean live Gold in a pre-copy action. Copy must finish before publication; a failed/partial copy leaves the prior Gold unchanged. Serial execution and one writer are required. The two publications are separate transactions, not atomic across both datasets. A transfer left by an earlier failed run is not permission to publish. The expected files do not establish end-to-end readiness.

Gold retains accepted order-line grain and device/day grain. Use SQL to shape consumer questions, for example:

```sql
SELECT order_date, customer_region, product_category, order_status,
       COUNT_BIG(*) AS accepted_lines,
       SUM(line_amount) AS accepted_line_amount
FROM gold.s04_lab_order_line
GROUP BY order_date, customer_region, product_category, order_status
ORDER BY order_date, customer_region, product_category, order_status;

SELECT reading_date, site_id, device_type,
       SUM(reading_count) AS accepted_readings,
       MIN(min_temperature_c) AS minimum_temperature_c,
       MAX(max_temperature_c) AS maximum_temperature_c
FROM gold.s04_lab_device_daily
GROUP BY reading_date, site_id, device_type
ORDER BY reading_date, site_id, device_type;
```

Do not average rounded device/day averages to produce a wider event mean. The current contract deliberately does not publish an exact sum; extend it with the appropriate sum/count if that consumer requirement is approved.

Completion evidence

Retain the source scope/version; full-data profiles; labeled Wrangler sample and reviewed export; three charts with grain/denominators; an explained Silver allowlist; all acceptance/rejection counts; complete expected-row comparisons; successful copy; Gold batch/count/amount checks; and intended-reader/model freshness evidence.

Rerun unchanged inputs without concurrent writers and compare **all business rows in both directions**, excluding ingestion/publication timestamps. Discuss a failed-copy case and preserve the previous Gold; do not manufacture a successful handoff by typing expected data directly into Warehouse.

This lab is batch/current-snapshot processing, not streaming, incremental correction, production schema evolution, or proof of tenant availability. Offline validation does not execute Spark, Data Wrangler, Warehouse SQL, copy/connector paths, or consumer access. Record those steps as completed, demonstrated, prepared evidence, or blocked in the actual tenant.

Source: 09_realistic_data_lab.md. Companion-material links open the repository originals.